Dokumentasi Pembuatan Aplikasi eLMS dengan Python dan Django

---

1. Pendahuluan

Aplikasi **eLMS (E-Learning Management System)** dikembangkan untuk mendukung proses pembelajaran daring.
Sistem ini menyediakan fitur manajemen mata kuliah, keanggotaan siswa, konten pembelajaran, dan komentar antar pengguna.

Framework **Django** dipilih karena memiliki arsitektur **Model-View-Template (MVT)** yang terstruktur dengan baik serta dukungan **ORM (Object Relational Mapping)** untuk mempermudah pengelolaan database.

---

2. Persiapan Lingkungan Pengembangan

Langkah awal sebelum membangun proyek:

1. Instal **Python 3.12** atau versi terbaru.
2. Buat virtual environment:
3. `python -m venv env`
4. Aktifkan environment:
    - Windows: `env\Scripts\activate`
    - Linux/Mac: `source env/bin/activate`
5. Instal Django:
6. `pip install django`

---

3. Membuat Proyek Django

Buat proyek utama dengan perintah:

```
django-admin startproject eLMS_project
```

Masuk ke direktori proyek:

```
cd eLMS_project
```

---

4. Membuat Aplikasi Inti (coreapp)

Buat aplikasi baru di dalam proyek:

```
python manage.py startapp coreapp
```

Kemudian tambahkan `'coreapp'` ke dalam daftar `INSTALLED_APPS` di file `settings.py`.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # local app
    'coreapp',
]
```

---

5. Membuat Model Database

Buka file `coreapp/models.py` dan tambahkan model utama seperti berikut:

```
class Course(models.Model):
    name = models.CharField("nama matkul", max_length=100)
    description = models.TextField("deskripsi", default='-')
    price = models.IntegerField("harga", default=10000)
    teacher = models.ForeignKey(User, on_delete=models.RESTRICT)
    created_at = models.DateTimeField(auto_now_add=True)
```

Model lain yang saling berhubungan:

- **CourseMember** (anggota kelas)
- **CourseContent** (materi pembelajaran)
- **Comment** (komentar siswa dan asisten)

```python
from django.db import models
from django.contrib.auth.models import User

class Course(models.Model):
    name = models.CharField("nama matkul", max_length=100)
    description = models.TextField("deskripsi", default='-')
    price = models.IntegerField("harga", default=10000)
    image = models.ImageField("gambar", null=True, blank=True)
    teacher = models.ForeignKey(User, verbose_name="pengajar",
on_delete=models.RESTRICT)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Mata Kuliah"
        verbose_name_plural = "Mata Kuliah"

    def __str__(self) -> str:
        return self.name + " : " + str(self.price)


ROLE_OPTIONS = [('std', "Siswa"), ('ast', "Asisten") ]

class CourseMember(models.Model):
    course_id = models.ForeignKey(Course, verbose_name="matkul",
on_delete=models.RESTRICT)
    user_id = models.ForeignKey(User, verbose_name="siswa",
on_delete=models.RESTRICT)
    roles = models.CharField("peran", max_length=3, choices=ROLE_OPTIONS,
default='std')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Subscriber Matkul"
        verbose_name_plural = "Subscriber Matkul"
        unique_together = ('course_id', 'user_id')

    def __str__(self) -> str:
        return str(self.course_id) + " : " + str(self.user_id)


class CourseContent(models.Model):
    name = models.CharField("judul konten", max_length=200)
    description = models.TextField("deskripsi", default='-')
```

```
    video_url = models.CharField('URL Video', max_length=200, null=True,
blank=True)
    file_attachment = models.FileField("File", null=True, blank=True)
    course_id = models.ForeignKey(Course, verbose_name="matkul",
on_delete=models.RESTRICT)
    parent_id = models.ForeignKey("self", verbose_name="induk",
                                  on_delete=models.RESTRICT, null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Konten Matkul"
        verbose_name_plural = "Konten Matkul"

    def __str__(self) -> str:
        return '[' + str(self.course_id) + "] " + self.name


class Comment(models.Model):
    content_id = models.ForeignKey(CourseContent, verbose_name="konten",
on_delete=models.CASCADE)
    member_id = models.ForeignKey(CourseMember, verbose_name="pengguna",
on_delete=models.CASCADE)
    comment = models.TextField('komentar')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Komentar"
        verbose_name_plural = "Komentar"

    def __str__(self) -> str:
        return "Komen: " + self.content_id.name + "-" +
self.member_id.user_id.username
```

---

6. Migrasi Database

Setelah model dibuat, jalankan perintah berikut untuk membuat tabel database:

```
python manage.py makemigrations
python manage.py migrate
```

---

7. Membuat View dan URL

Tambahkan fungsi view di `coreapp/views.py`:

```
def courses(request):
    data = Course.objects.all()
    return render(request, 'courses.html', {'courses': data})
```

```
from django.shortcuts import render, get_object_or_404, redirect
from django.contrib.auth.decorators import login_required
from django.views.generic import ListView, DetailView
from django.contrib import messages
from django.contrib.auth.models import User
from .models import Course, CourseMember, CourseContent, Comment

def core(request):
    return render(request, 'core.html')

def users(request):
    myusers = User.objects.all()
    context = {'myusers': myusers}
    return render(request, 'all_users.html', context)

def courses(request):
    all_courses = Course.objects.all()
    context = {'courses': all_courses}
    return render(request, 'all_courses.html', context)

class CourseListView(ListView):
    model = Course
    template_name = 'course_list.html'
    context_object_name = 'courses'
    ordering = ['-created_at']

class CourseDetailView(DetailView):
    model = Course
    template_name = 'course_detail.html'
    context_object_name = 'course'


@login_required
def join_course(request, pk):
    course = get_object_or_404(Course, pk=pk)
    member, created = CourseMember.objects.get_or_create(
        course_id=course,
        user_id=request.user,
```

```
        defaults={'roles': 'std'}
    )
    if created:
        messages.success(request, f"Kamu berhasil bergabung di {course.name}")
    else:
        messages.info(request, f"Kamu sudah terdaftar di {course.name}")
    return redirect('course_detail', pk=pk)


@login_required
def course_content_list(request, course_id):
    course = get_object_or_404(Course, id=course_id)
    contents = CourseContent.objects.filter(course_id=course)
    return render(request, 'course_content_list.html', {
        'course': course,
        'contents': contents
    })


@login_required
def course_content_detail(request, content_id):
    content = get_object_or_404(CourseContent, id=content_id)
    comments = Comment.objects.filter(content_id=content)
    return render(request, 'course_content_detail.html', {
        'content': content,
        'comments': comments
    })


@login_required
def add_comment(request, content_id):
    content = get_object_or_404(CourseContent, id=content_id)
    member = CourseMember.objects.filter(course_id=content.course_id,
user_id=request.user).first()

    if not member:
        messages.error(request, "Kamu belum tergabung di mata kuliah ini.")
        return redirect('course_detail', pk=content.course_id.id)

    if request.method == 'POST':
        text = request.POST.get('comment')
        if text:
            Comment.objects.create(content_id=content, member_id=member,
comment=text)
            messages.success(request, "Komentar berhasil ditambahkan.")
```

```
        return redirect('course_content_detail', content_id=content.id)

    return render(request, 'add_comment.html', {'content': content})


@login_required
def my_courses(request):
    memberships = CourseMember.objects.filter(user_id=request.user)
    return render(request, 'my_courses.html', {'memberships': memberships})
```

Daftarkan URL di `eLMS_project/urls.py`:

`path('courses/', views.courses),`

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.core, name='core'),
    path('users/', views.users, name='users'),
    path('courses/', views.courses, name='courses'),
    path('courses/list/', views.CourseListView.as_view(), name='course_list'),
    path('course/<int:pk>/', views.CourseDetailView.as_view(),
name='course_detail'),
    path('course/<int:pk>/join/', views.join_course, name='join_course'),
    path('course/<int:course_id>/contents/', views.course_content_list,
name='course_content_list'),
    path('content/<int:content_id>/', views.course_content_detail,
name='course_content_detail'),
    path('content/<int:content_id>/comment/', views.add_comment,
name='add_comment'),
    path('my-courses/', views.my_courses, name='my_courses'),
]
```

---

8. Menambahkan Template

Buat folder `templates` di dalam `coreapp`.
Buat file `courses.html` untuk menampilkan daftar mata kuliah:

```
<h1>Daftar Mata Kuliah</h1>
<ul>
{% for course in courses %}
```

```
    <li>{{ course.name }} - Rp{{ course.price }}</li>
{% endfor %}
</ul>
```

---

9. Menjalankan Server

Gunakan perintah berikut untuk menjalankan server lokal:

```
python manage.py runserver
```

Akses di browser:
👉 `http://127.0.0.1:8000/`

---

10. CRUD dan Admin

Gunakan Django Admin untuk operasi CRUD.
Daftarkan model di `coreapp/admin.py`:

```
from .models import Course
admin.site.register(Course)
```

Akses panel admin:

```
http://127.0.0.1:8000/admin
```

---

11. Login dan Autentikasi

Gunakan sistem login bawaan Django (`django.contrib.auth`).
Tambahkan halaman login:

```
from django.contrib.auth import authenticate, login

def user_login(request):
    ...
```

Bisa juga menggunakan template bawaan:
`{% if user.is_authenticated %}` untuk memeriksa status login.

---

12. Deployment dengan Nginx (Opsional)

Gunakan **Gunicorn** dan **Nginx** untuk deployment di server Linux.

Contoh konfigurasi Nginx:

```
server {
    listen 80;
    server_name domainkamu.com;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
    }
}
```

---

13. Kesimpulan

Proyek eLMS berhasil dibuat menggunakan Django dengan fitur utama:

- Manajemen pengguna dan mata kuliah
- CRUD berbasis admin
- Relasi antar tabel dengan ORM
- Sistem login & autentikasi
- Dapat dikembangkan menjadi sistem e-learning lengkap dengan video, kuis, dan forum diskusi

---